Use Depth First Search For problem solving maze solving

```
def dfs_maze(grid, start, goal):
    rows = len(grid)
    cols = len(grid[0])

    visited = set()
    path = []

    def dfs(r, c):
        # Check boundaries
        if not (0 <= r < rows and 0 <= c < cols):
            return False

        # Check wall or visited cell
        if grid[r][c] == 1 or (r, c) in visited:
            return False

        # Mark cell as visited
        visited.add((r, c))
        path.append((r, c))

        # Check if goal is reached
        if (r, c) == goal:
            return True

        # Down, Right, Up, Left
        directions = [
            (1, 0),
            (0, 1),
```

```
            (-1, 0),
            (0, -1)
        ]

        for dr, dc in directions:
            if dfs(r + dr, c + dc):
                return True

        # Backtracking
        path.pop()
        return False

    if dfs(start[0], start[1]):
        return path
    else:
        return None


# 0 = Open path
# 1 = Wall

maze = [
    [0, 1, 0, 0, 0],
    [0, 1, 0, 1, 0],
    [0, 0, 0, 1, 0],
    [1, 1, 0, 0, 0],
    [0, 0, 0, 1, 0]
]

# Starting and Goal positions
start = (0, 0)
```

```
goal = (4, 2)

# Find DFS path
path = dfs_maze(maze, start, goal)

# Display output
print("DFS path:", path)
print("Path found:", path is not None)
#run
dfs_maze.py
```